

```
krishnamohan_koyya@instance-1:~$ kube get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kafka	NodePort	10.107.238.13	<none>	9092:30092/TCP	2d13h
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	32d
mysqlpdb	ClusterIP	None	<none>	3306/TCP	2d13h
ums-add-service	LoadBalancer	10.99.227.184	10.99.227.184	8080:30961/TCP	2d13h
ums-find-service	LoadBalancer	10.98.45.187	10.98.45.187	8080:31649/TCP	2d13h
zookeeper	NodePort	10.102.24.155	<none>	2181:30181/TCP	2d13h

Figure 3: Kubernetes services

```
krishnamohan_koyya@instance-1:~$ curl http://10.98.45.187:8080/user/KrishnaMohan
{"name":"KrishnaMohan","phone":9731423166,"since":"Sat, 09 Jul 2022 08:04:30 GMT"}
```

Figure 4: FindService

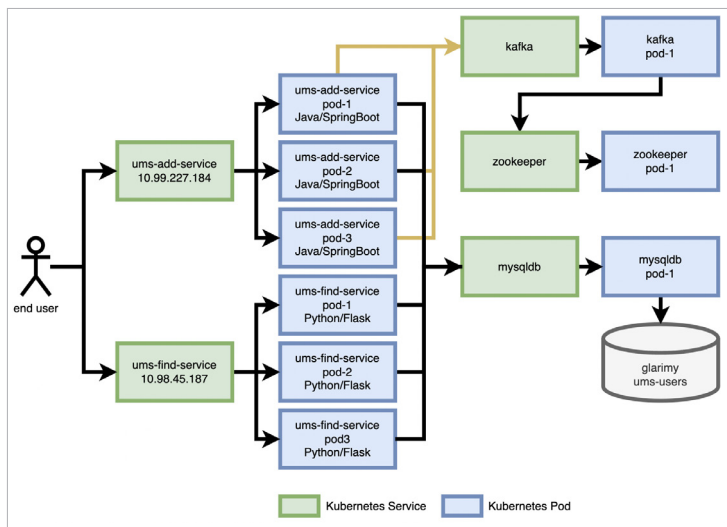


Figure 5: UMS with AddService and FindService

deployment of *FindService* from the image *glarimy/ums-find-service* with three replicas. It also exposes port 8080. And the latter part of the manifest declares a Kubernetes service as the front-end for the *FindService* deployment. Recollect that the MySQL service in the name of *mysqlpdb* was already part of the above manifest from the previous part.

Run the following command to deploy the manifest on a Kubernetes cluster:

```
kubectl create -f kube-find-deployment.yml
```

Once the deployment is finished, you can verify the pods and services using the following command:

```
kubectl get services
```

It gives an output as shown in Figure 3.

It lists all the services running on the cluster. Make a note of the external-ip of the *FindService* and use the *curl*

command to invoke the service:

```
curl http://10.98.45.187:8080/user/KrishnaMohan
```

Note that the IP address 10.98.45.187 corresponds to the *FindService*, as found in Figure 3. If we have used the *AddService* to create a user by the name KrishnaMohan, the above *curl* command looks like what is shown in Figure 4.

With the *AddService* and *FindService*, along with the required back-end services for storage and messaging, the architecture of the user management system (UMS) stands as shown in Figure 5, at this point. You can observe that the end-user uses the IP address of the *ums-add-service* for adding a new user, whereas it uses the IP address of the *ums-find-service* for finding existing users. Each of these Kubernetes services are backed by three pods with the corresponding containers. Also note that the same *mysqlpdb* service is used for storing and retrieving the user data.

What about the other services?

The UMS consists of two more services, namely, *SearchService* and *JournalService*. We will design these services in the next part of this series on the Node platform, and deploy them on the same Kubernetes cluster to demonstrate the real power of polyglot microservices architecture. We will conclude by observing some design patterns pertaining to microservices. **END** 🐧

By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.